

Python Programming Notes

Theory and Concepts

Units I – V

Unit I · Introduction to Python and Control Flow

1.1 Introduction to Python

Python is a high-level, general-purpose programming language known for its simple syntax and readability. It supports procedural, object-oriented and functional programming styles. Python programs are generally shorter and easier to understand than equivalent programs written in many lower-level languages.

Features of Python

- **Simple and readable:** Python uses clear syntax and indentation, making programs easier to read.
- **High-level language:** It hides many low-level implementation details from the programmer.
- **Interpreted execution:** Python source code is executed by the Python interpreter.
- **Portable:** The same Python program can usually run on different operating systems with little or no modification.
- **Object-oriented:** Python supports classes, objects, inheritance, polymorphism and abstraction.
- **Dynamically typed:** A variable does not need an explicit data type declaration before use.
- **Large library support:** Python provides a rich standard library and many external packages.
- **Open source:** Python is freely available and has a large developer community.

History and Applications

Python was created by Guido van Rossum and first released in the early 1990s. It has evolved into a widely used language for software development, automation, web development, data analysis, scientific computing, artificial intelligence and education.

1.2 Python IDEs

An **Integrated Development Environment (IDE)** provides tools for writing, running and debugging programs. Common Python development environments include IDLE, Visual Studio Code, PyCharm and Jupyter environments. An IDE normally provides a code editor, execution support, error reporting and debugging facilities.

1.3 Python Building Blocks

Indentation

Indentation means the spaces at the beginning of a line. Python uses indentation to define a block of statements instead of braces. Statements belonging to the same block must use consistent indentation.

```
if marks >= 40:  
    print("Pass")  
else:  
    print("Fail")
```

Identifiers

An identifier is the name given to a variable, function, class, module or other program element. It can contain letters, digits and underscores, but it cannot begin with a digit. Python identifiers are case-sensitive.

Variables

A variable is a name that refers to a value. The type of the value is determined at runtime.

```
age = 20
name = "Asha"
marks = 78.5
```

Constants

Python does not enforce constants through a special keyword. By convention, names intended to remain unchanged are written in uppercase, such as **PI = 3.14159**.

Keywords

Keywords are reserved words with predefined meanings in Python, such as **if, else, for, while, class, def, return, import** and **True**. They cannot be used as ordinary identifiers.

1.4 Basic Input and Output

input() reads data from the user and returns it as a string. Conversion functions can be used when numeric data is required. **print()** displays information on the screen.

```
name = input("Enter name: ")
age = int(input("Enter age: "))
print("Name:", name)
print("Age:", age)
```

1.5 Operators

Arithmetic Operators

Arithmetic operators perform mathematical calculations: **+**, **-**, *****, **/**, **//** (floor division), **%** (remainder) and ****** (power).

Relational Operators

Relational operators compare values and produce a Boolean result: **==**, **!=**, **>**, **<**, **>=**, **<=**.

Assignment Operators

Assignment operators store or update values. Examples include **=**, **+=**, **-=**, ***=**, **/=**, **//=**, **%=** and ****=**.

Logical Operators

and returns true when both conditions are true; **or** returns true when at least one condition is true; **not** reverses a Boolean result.

Bitwise Operators

Bitwise operators work on the binary representation of integers. Important operators are **&**, **|**, **^**, **~**, **<<**, **>>**.

1.6 Control Flow Statements

Conditional Statements

Conditional statements allow a program to choose different paths based on a condition. Python provides **if**, **if-else** and **if-elif-else**.

```
if marks >= 75:
    grade = "A"
elif marks >= 60:
    grade = "B"
else:
    grade = "C"
```

Nested if

A nested if is an if statement placed inside another if statement. It is useful when a second condition should be checked only after a first condition is satisfied.

Loops

Loops repeat a block of statements. A **for** loop is commonly used when iterating over a sequence or a range. A **while** loop repeats while its condition remains true.

```
for i in range(1, 6):  
    print(i)  
  
count = 1  
while count <= 5:  
    print(count)  
    count += 1
```

Loop Control Statements

break terminates the nearest loop. **continue** skips the remaining statements of the current iteration and proceeds to the next iteration. **pass** is a null statement used where syntactically a statement is required but no action is needed.

Unit II · Data Structures in Python

2.1 List

A **list** is an ordered, mutable collection. It can contain elements of different data types and allows duplicate values. List indexing starts at 0.

Creating and Accessing Lists

```
numbers = [10, 20, 30, 40]
print(numbers[0])
print(numbers[-1])
```

Updating and Deleting Values

Because lists are mutable, individual elements can be changed. Elements can be removed using operations such as **del**, **remove()** and **pop()**.

```
numbers[1] = 25
numbers.append(50)
numbers.remove(30)
last = numbers.pop()
```

Basic List Operations

- **Concatenation (+)**: joins two lists.
- **Repetition (*)**: repeats a list.
- **Membership (in / not in)**: checks whether an element exists.
- **Slicing**: extracts a portion of a list using start, stop and step positions.

Built-in List Functions and Methods

Useful functions and methods include **len()**, **max()**, **min()**, **sum()**, **append()**, **extend()**, **insert()**, **remove()**, **pop()**, **sort()**, **reverse()** and **clear()**.

2.2 Tuple

A **tuple** is an ordered, immutable collection. It is created using parentheses, although parentheses are not always required. Tuples support indexing and slicing but their elements cannot be changed after creation.

```
point = (10, 20)
print(point[0])
```

Basic Tuple Operations

Tuples support concatenation, repetition, membership testing and slicing. Common tuple methods are **count()** and **index()**. The **len()** function can be used to find the number of elements.

2.3 Set

A **set** is an unordered collection of unique elements. Duplicate values are automatically removed. Sets are useful for membership testing and mathematical set operations.

```
colors = {"red", "blue", "green"}
colors.add("yellow")
colors.discard("red")
```

Set Operations

Important set operations include **union**, **intersection**, **difference** and **symmetric difference**.

```
A = {1, 2, 3}
B = {3, 4, 5}
print(A | B) # union
print(A & B) # intersection
print(A - B) # difference
print(A ^ B) # symmetric difference
```

Built-in Set Methods

Common methods include **add()**, **update()**, **remove()**, **discard()**, **pop()**, **clear()**, **union()**, **intersection()** and **difference()**.

2.4 Dictionary

A **dictionary** stores data as key-value pairs. Keys must be unique and hashable, while values may be of different types. Dictionaries are mutable.

```
student = {"name": "Riya", "marks": 82}
print(student["name"])
student["marks"] = 88
student["city"] = "Pune"
```

Accessing, Updating and Deleting

Values can be accessed using keys or methods such as **get()**. Existing values can be updated by assigning a new value to a key. A key-value pair can be removed using **del**, **pop()** or **popitem()**.

Basic Dictionary Operations

Useful operations include membership testing with **in**, obtaining keys with **keys()**, values with **values()**, and key-value pairs with **items()**. The **update()** method can add or modify multiple entries.

Built-in Dictionary Methods

Important methods include **get()**, **keys()**, **values()**, **items()**, **update()**, **pop()**, **popitem()**, **setdefault()** and **clear()**.

Unit III · Functions, Modules and Packages in Python

3.1 Functions

A **function** is a reusable block of code designed to perform a particular task. Functions improve modularity, readability, testing and code reuse.

Defining and Calling a Function

```
def add(a, b):  
    return a + b  
  
result = add(10, 20)  
print(result)
```

Parameters and Arguments

A parameter is a variable listed in a function definition. An argument is the actual value passed when the function is called. Python supports positional, keyword, default and variable-length arguments.

Return Statement

The **return** statement sends a value back to the caller and ends the function execution at that point. A function without an explicit return statement returns **None**.

Scope of Variables

The scope of a variable determines where it can be accessed. A local variable is defined inside a function and is normally available only within that function. A global variable is defined outside functions and can be accessed according to Python's name-resolution rules.

Lambda Functions

A lambda is a small anonymous function written using the **lambda** keyword. It is generally used for short expressions.

```
square = lambda x: x * x  
print(square(5))
```

3.2 Modules

A **module** is a Python file containing definitions such as functions, classes and variables. Modules help organize programs into separate, reusable components.

Creating and Importing a User-Defined Module

A module can be created as a normal Python file and imported using **import**. Specific members can also be imported with **from ... import ...**.

```
import math  
print(math.sqrt(25))  
  
from math import factorial  
print(factorial(5))
```

Built-in Modules

Python includes many useful modules such as **math** for mathematical operations, **random** for random values, **datetime** for date and time handling, and **os** for operating-system-related tasks.

Namespace and Scope

A namespace is a mapping between names and objects. Python maintains different namespaces for built-in names, global names and local names. Scope determines which namespaces are searched when a name is used.

3.3 Python Packages

A **package** is a way of organizing related modules in a directory structure. Packages help manage larger programs by grouping related functionality.

Creating and Importing a Package

A user-defined package can contain one or more modules. Its modules can be imported using dotted names such as **package.module**.

Built-in and External Packages

Python's ecosystem contains packages for many domains. Standard-library modules are included with Python, while external packages are installed separately and can provide specialized functionality.

Installing Packages

The Python Package Installer, commonly called **pip**, is used to install packages from package repositories. A typical command is:

```
pip install package_name
```


Unit IV · Object-Oriented Programming in Python

4.1 Object-Oriented Concepts

Object-Oriented Programming (OOP) organizes software around **objects** that combine data and behavior. Python supports OOP through classes, objects, methods, inheritance and other related concepts.

Class

A **class** is a blueprint for creating objects. It defines attributes and methods that describe the state and behavior of its objects.

Object

An **object** is an instance of a class. Different objects created from the same class can hold different attribute values.

```
class Student:
    def show(self):
        print("Student object")

s1 = Student()
s1.show()
```

4.2 Constructors in Python

A constructor-like initialization method is written as **__init__()**. It is called automatically when an object is created and is commonly used to initialize instance attributes.

Non-Parameterized Constructor

```
class Student:
    def __init__(self):
        self.name = "Unknown"
```

Parameterized Constructor

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
```

self Parameter

self refers to the current object instance. It is used to access instance attributes and methods. It is written explicitly in method definitions, although it is supplied automatically when the method is called through an object.

4.3 Polymorphism

Polymorphism means one interface can be used with different forms of objects. In Python, polymorphism commonly appears through method overriding and duck typing.

Method Overriding

When a child class provides its own implementation of a method already defined in the parent class, the child implementation overrides the inherited version.

```
class Animal:
    def sound(self):
        print("Some sound")

class Dog(Animal):
    def sound(self):
```

```
print("Bark")
```

Method Overloading

Traditional compile-time method overloading is not implemented in Python in the same way as in some statically typed languages. Similar behavior can be achieved using default arguments, variable-length arguments or conditional logic.

4.4 Data Hiding and Abstraction

Data Hiding

Data hiding means restricting direct access to internal implementation details. Python uses naming conventions such as a leading underscore and name mangling for double-underscore attributes to indicate or support restricted access.

Abstraction

Abstraction focuses on what an object does rather than how it performs the task. Python can implement abstraction using abstract base classes and the **abc** module.

4.5 Inheritance

Inheritance allows a new class to reuse and extend features of an existing class. The existing class is commonly called the parent or base class, and the new class is called the child or derived class.

Single Inheritance

One child class inherits from one parent class.

Multiple Inheritance

One child class inherits from more than one parent class.

Multilevel Inheritance

Inheritance occurs across multiple levels, such as a grandparent class → parent class → child class.

```
class A:
    pass

class B(A):
    pass

class C(B):
    pass
```

Unit V · Introduction to Built-in Packages in Python

5.1 Pandas

Pandas is a Python library widely used for data manipulation and analysis. Its two fundamental data structures are **Series** and **DataFrame**.

Series

A Series is a one-dimensional labeled data structure. It can hold values of different compatible types and provides an index for accessing data.

```
import pandas as pd
s = pd.Series([10, 20, 30])
print(s)
```

DataFrame

A DataFrame is a two-dimensional labeled table with rows and columns. It is commonly used for structured datasets.

```
import pandas as pd
data = {
    "Name": ["A", "B", "C"],
    "Marks": [75, 82, 68]
}
df = pd.DataFrame(data)
print(df)
```

Reading CSV Files

Pandas can read comma-separated values files using **read_csv()**. The resulting DataFrame can then be inspected, filtered, transformed and analyzed.

```
df = pd.read_csv("students.csv")
print(df.head())
```

5.2 Creating GUI Applications Using Tkinter

Tkinter is Python's standard GUI toolkit. It provides widgets and mechanisms for creating desktop graphical interfaces.

Important Tkinter Widgets

- **Label:** displays text or information.
- **Entry:** accepts single-line text input.
- **Button:** performs an action when clicked.
- **Radiobutton:** lets the user select one option from a group.
- **Checkbutton:** represents an independent on/off selection.

Basic GUI Structure

A Tkinter application normally creates a root window, places widgets in the window, connects widget events to functions and starts the event loop.

```
import tkinter as tk

root = tk.Tk()
root.title("Sample GUI")

tk.Label(root, text="Hello").pack()
```

```
tk.Button(root, text="Close", command=root.destroy).pack()

root.mainloop()
```

Layout Managers

Tkinter provides layout managers such as **pack()**, **grid()** and **place()**. They control how widgets are arranged within a window.

5.3 Connecting to a Database Using MySQL

Python can connect to a MySQL database using an appropriate database connector. A typical database interaction involves connecting to the server, creating a cursor, executing SQL statements, retrieving results and closing the connection.

Basic Connection Steps

- Install an appropriate MySQL connector package.
- Create a connection using host, username, password and database information.
- Create a cursor object to execute SQL statements.
- Execute SQL commands such as SELECT, INSERT, UPDATE or DELETE.
- Fetch results when a query returns data.
- Commit changes for operations that modify data.
- Close the cursor and database connection.

```
import mysql.connector

con = mysql.connector.connect(
    host="localhost",
    user="root",
    password="password",
    database="college"
)

cursor = con.cursor()
cursor.execute("SELECT * FROM students")
rows = cursor.fetchall()

for row in rows:
    print(row)

cursor.close()
con.close()
```

Note: Database applications should validate input, handle connection/query errors appropriately, and avoid building SQL statements by directly concatenating untrusted input. Parameterized queries are preferred.

Quick Concept Revision

Topic	Key Point
Python	High-level, readable, general-purpose programming language.
Indentation	Defines blocks of Python statements.
List	Ordered and mutable collection; duplicates allowed.
Tuple	Ordered and immutable collection.
Set	Unordered collection of unique elements.
Dictionary	Mutable collection of key-value pairs.
Function	Reusable block of code for a specific task.
Module	Python file containing reusable definitions.
Package	Collection used to organize related modules.
Class	Blueprint for creating objects.
Object	Instance of a class.
Inheritance	Mechanism for reusing and extending class features.
Polymorphism	Same interface/operation can work with different object types.
Pandas	Library for data manipulation and analysis.
Tkinter	Standard Python toolkit for desktop GUIs.
MySQL Connector	Used to communicate between Python programs and MySQL databases.